INTELLIGENT VEHICLE FUEL CONSUMPTION AND EFFICIENCY ANALYTICS SYSTEM USING PYTHON

REPORT

CSA0806– PYTHON PROGRAMMING

Course Outcomes: CO1,CO2, CO3 and CO4

Submitted by

Name: RASHMIKA R

Register No.: 192519002

Department: BME

Year / Semester: 2

Submitted to

Faculty Name: Dr. S. Ponmaniraj

Institution: SIMATS Engineering.

Academic Year: 2026–2027

CERTIFICATE

This is to certify that **RASHMIKA R** of **BIOMEDICAL ENGINEERING** has successfully prepared and submitted the report entitled **"INTELLIGENT VEHICLE FUEL CONSUMPTION AND EFFICIENCY ANALYTICS SYSTEM USING PYTHON"** as part of the academic requirements for CSA0806-Python programming. The work presented in this report addresses Python-based Vehicle Fuel Management System provides a menu-driven solution to register vehicles, record fuel and trip data, calculate mileage and monthly consumption, analyse fuel costs, rank vehicle efficiency, detect abnormal trips, and generate a consolidated report.

Faculty Signature: _____

Date: _____

TABLE OF CONTENTS

ABSTRACT

Fuel is one of the biggest recurring costs that vehicle owners and small fleet operators have to deal with. The trouble is, most people still track it the old-fashioned way — a notebook here, a spreadsheet there — and once the records get scattered like that, it's genuinely hard to keep them consistent or pull any real insight out of them. The Vehicle Fuel Consumption Tracking and Analysis System was built to fix exactly this gap. It's a menu-driven Python application that lets a user register vehicles, log refuelling and trip details, and then turns those raw entries into performance indicators that actually mean something.

Under the hood, the system follows a modular design, with separate components handling input validation, data access, analysis and report generation. Core Python data structures — lists, dictionaries, tuples and sets — are used deliberately throughout, partly to keep the code clean and partly to show a solid grasp of when each structure actually fits. CSV files handle persistence, keeping storage simple. From there, the application works out distance travelled using odometer readings, mileage in kilometres per litre, monthly fuel consumption, estimated cost, and an efficiency ranking across vehicles, and it flags any trip whose fuel efficiency drops below a chosen threshold.

Beyond the calculations themselves, the project shows how a handful of basic programming concepts, put together thoughtfully, can turn into a genuinely useful information system. It's also built with room to grow — database storage, a graphical dashboard, live fuel-price integration, GPS/telematics data and predictive analytics are all natural next steps. For now, though, the scope is deliberately kept to individuals and small fleets, which keeps it realistic as a command-line academic project while still producing output that says something meaningful.

1. Problem Statement and Problem Formulation

1.1 Background

Of all the costs tied to running a vehicle, fuel is one of the easiest to actually measure. For someone driving a personal car, that cost shows up directly in household travel spending. For a small fleet, it adds up fast — multiply it across several vehicles and hundreds of trips, and even a small dip in efficiency can leave a real dent in the monthly budget.

Most people already keep some record of what they spend on fuel, but it's usually scattered — a receipt here, a line in a notebook, a row in a spreadsheet. The problem is that a receipt tells you what you paid, not how far you actually drove since the last fill-up. Without some structure behind these numbers, basic questions become surprisingly hard to answer: which vehicle is the most efficient, which one is eating the most fuel, was that one trip actually unusual, and how does spending shift from one month to the next?

1.2 Problem Statement

What individuals and small fleet operators really need is something simple — a way to register their vehicles, log fuel and trip details, and get fuel-consumption metrics and efficiency figures without doing the arithmetic by hand every time. Just as important, the system has to hold onto that history so it can be looked back on and analysed later.

1.3 Problem Formulation

In practice, the problem breaks down into a chain of steps: capturing data, validating it, storing it, transforming it, and finally analysing it. A record only counts as valid once it has a vehicle identifier along with trip or fuel information attached. Where both a starting and an ending odometer reading are available, those two numbers are what determine the distance travelled.

Metric	Formula	Purpose
Distance	End Odometer − Start Odometer	Determine kilometres travelled.
Mileage	Distance ÷ Fuel	Measure vehicle efficiency in km/L.
Fuel Cost	Fuel × Fuel Price	Estimate expenditure for a record.
Monthly Fuel	Sum of fuel records for a month	Measure monthly consumption.
Average Monthly Fuel	Total fuel ÷ number of months	Summarise consumption over a period.

1.4 Scope

- Vehicle registration and basic vehicle information management.
- Fuel/trip record entry and retrieval.
- Input validation and basic error handling.
- Fuel-consumption and mileage calculations.

- Monthly grouping and summary.
- Vehicle efficiency ranking.
- Abnormal-trip identification.
- CSV persistence and consolidated text report generation.

1.5 Significance

The value of this project isn't just in spitting out a single mileage number. What it really creates is a repeatable workflow — raw entries go in, get validated and stored, and come out the other end as comparable information. That's what makes it useful for real decisions: spotting an inefficient vehicle, double-checking a trip that looks off, or getting a rough sense of what next month's fuel budget might look like.

2. Objective and Expected Outcomes

2.1 Objectives

- Develop a menu-driven Python application for vehicle fuel and trip tracking.
- Register vehicles with unique vehicle identifiers and basic descriptive information.
- Record trip and refuelling information in a structured format.
- Validate vehicle numbers, dates, units, odometer readings, fuel quantities and prices.
- Calculate travelled distance from odometer readings.
- Calculate mileage in kilometres per litre.
- Calculate monthly fuel consumption and summary statistics.
- Estimate fuel cost using user-provided fuel-price values.
- Use Python collections for grouping, filtering, summarisation and duplicate handling.
- Rank vehicles by fuel efficiency.
- Identify trips with abnormally high fuel consumption.
- Persist data in CSV files.
- Use custom modules to separate concerns.
- Generate a consolidated report for decision-making.

2.2 Expected Outcomes

Once finished, the system should take raw vehicle and fuel entries and turn them into a proper set of analytical outputs. In practice, that means a user can go from registering a vehicle and entering trip data straight to a meaningful summary, without ever touching a calculator.

Expected Outcome	Evidence
Persistent records	Vehicles and trips reload successfully after program restart.
Correct mileage	Program output agrees with manually calculated sample values.
Monthly analysis	Records are grouped correctly by month.
Efficiency ranking	Vehicles appear in the correct descending order of mileage.
Abnormal-trip detection	Trips below the defined threshold are flagged.
Consolidated report	Final report includes totals, rankings and flagged trips.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

ID	Requirement	Description
FR1	Vehicle registration	Create and store a vehicle record.
FR2	Vehicle listing	Display stored vehicles.
FR3	Trip/fuel entry	Record date, odometer, distance, fuel and price.
FR4	Validation	Reject invalid or inconsistent input.
FR5	Mileage	Calculate km/L for valid records.
FR6	Monthly analysis	Group records by month and summarise fuel use.
FR7	Cost analysis	Estimate cost from fuel quantity and price.
FR8	Grouping	Group trip records by vehicle.
FR9	Deduplication	Identify duplicate identifiers/records.
FR10	Ranking	Sort vehicles by efficiency.
FR11	Abnormal detection	Flag unusually low mileage.
FR12	Reporting	Generate a consolidated report.
FR13	Persistence	Read/write CSV records.

3.2 Non-Functional Requirements

- Usability: menu options should be clearly labelled and outputs should be readable.
- Maintainability: functions should have focused responsibilities and modules should be logically separated.
- Reliability: invalid inputs should not terminate the program unexpectedly.
- Portability: the application should run using a standard Python 3.x installation.
- Performance: analysis should complete quickly for the intended small-fleet dataset size.
- Data integrity: invalid numeric and inconsistent odometer values should be rejected where appropriate.

3.3 Constraints

- CSV is used instead of a database in the initial version.

- The interface is command-line based.
- Fuel prices are entered by the user rather than automatically retrieved.
- The system is designed for small datasets.
- No live GPS, telematics or vehicle-sensor integration is included.
- Results depend on the accuracy of the records entered.

3.4 Assumptions

- Distance is recorded in kilometres.
- Fuel quantity is recorded in litres.
- Fuel price is recorded in ₹ per litre.
- Vehicle numbers are unique.
- Trip IDs are unique.
- Valid odometer readings do not decrease between the start and end of a trip.
- Fuel quantity is positive for a fuel-consumption calculation.

4. Application of Relevant Course Knowledge / Concepts

This project deliberately leans on the core concepts covered in a typical Python programming course — nothing is included just to tick a box. Each concept earns its place by doing actual work somewhere in the system.

Concept	Application	Example
Variables	Store input and calculated values.	fuel = 20.0
Data types	Represent numeric, textual and date values.	float, str, date
Strings	Clean and validate user-entered identifiers.	strip(), upper()
Lists	Maintain collections of records.	trips = []
Dictionaries	Represent structured records.	trip['fuel']
Tuples	Represent fixed summaries.	(vehicle_no, mileage)
Sets	Maintain unique identifiers.	set(vehicle_numbers)
if/elif/else	Validation and decision rules.	if fuel <= 0
Loops	Menu repetition and record processing.	while True
Functions	Separate responsibilities.	calculate_mileage()
Exceptions	Handle invalid input and file errors.	try/except
CSV	Persistent structured storage.	csv.DictReader
Modules	Separate validation/data/analysis/reporting.	import analysis
Sorting	Rank vehicles by efficiency.	sorted(..., key=...)
Filtering	Find abnormal trips.	if mileage < threshold
Aggregation	Calculate totals and averages.	sum(), grouped totals

4.1 Lists

Lists make sense here because the application needs an ordered collection that can keep growing as new records come in. A list of dictionaries turns out to be a straightforward way to hold every trip in memory.

```
trips = [
    {"trip_id": "1", "vehicle_no": "TN01AB1234", "distance": 250, "fuel": 20},
    {"trip_id": "2", "vehicle_no": "TN02CD5678", "distance": 180, "fuel": 25}
]
```

4.2 Dictionaries

Dictionaries earn their place because named fields are just easier to read than positional indexes. Rather than remembering that index 2 means ‘fuel’, the program can simply reach for keys such as vehicle_no, date, fuel and odometer.

4.3 Tuples

Tuples fit well for small summaries that shouldn't change shape once created — a ranking result, for instance, sits naturally as (vehicle number, mileage).

4.4 Sets

Sets store unique values and are therefore useful for finding unique vehicle numbers or checking whether an identifier has already appeared.

4.5 Functions and Modularisation

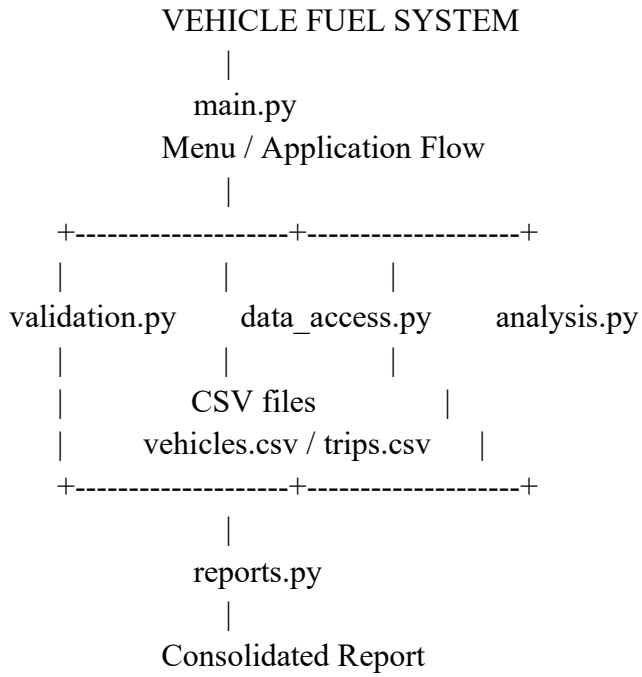
Functions cut down on repeated code and make testing much less painful. Following that same logic, the application splits input validation, CSV access, calculations and report formatting across different modules rather than piling everything into one place.

5. Design

5.1 Design Philosophy

The design that was ultimately chosen is modular, collection-oriented and built around CSV storage. It puts clarity first and stays close to the Python concepts the course actually covers, deliberately steering clear of a web server or a full database — while still leaving room to extend the system later if needed.

5.2 Proposed Architecture



5.3 Module Responsibilities

Module	Main Responsibility	Representative Functions
main.py	Menu and user interaction	main(), menu()
validation.py	Input cleaning and validation	validate_vehicle_number(), validate_date()
data_access.py	CSV persistence	load_vehicles(), save_vehicle(), load_trips(), save_trip()
analysis.py	Calculations and analytics	calculate_mileage(), rank_vehicles(), find_abnormal_trips()
reports.py	Formatted output	generate_report(), display_summary()

5.4 Data Model

There's no need for a relational database at this stage. Instead, every CSV row is simply treated as a record and loaded straight into a dictionary, and a list of those dictionaries becomes the working in-memory dataset.

5.5 Vehicle Data

Field	Type	Description	Validation
vehicle_no	String	Unique registration/vehicle identifier	Non-empty, cleaned, unique
model	String	Vehicle model	Non-empty
fuel_type	String	Petrol/Diesel/etc.	Non-empty
registration_date	String/Date	Registration date	Valid date

5.6 Trip Data

Field	Type	Description	Validation
trip_id	String/Integer	Unique trip identifier	Unique/non-empty
vehicle_no	String	Related vehicle	Must exist
date	String/Date	Trip/refuelling date	Valid date
start_odometer	Float	Starting reading	Non-negative
end_odometer	Float	Ending reading	>= starting reading
distance	Float	Calculated or entered distance	Positive
fuel_filled	Float	Fuel quantity	> 0
fuel_price	Float	Price per litre	> 0

5.7 Input-to-Report Methodology

- Capture input from the user.
- Clean and validate the input.
- Check vehicle/trip uniqueness and relationships.
- Calculate derived fields such as distance and mileage.
- Persist valid records to CSV.
- Load records into lists and dictionaries for analysis.
- Group records by vehicle and month.
- Calculate performance metrics.
- Sort vehicles for efficiency ranking.
- Apply abnormal-consumption rules.
- Format results into a consolidated report.

6. Algorithm

6.1 Main Algorithm

```
START
  Load vehicles.csv
  Load trips.csv
  WHILE True
    Display main menu
    Read choice
    IF choice = Register Vehicle
      Read vehicle details
      Validate details
      Save vehicle
    ELSE IF choice = Add Trip
      Read trip details
      Validate details
      Calculate distance
      Save trip
    ELSE IF choice = Mileage
      Calculate mileage
    ELSE IF choice = Monthly Analysis
      Group by month and calculate totals
    ELSE IF choice = Cost Analysis
      Calculate fuel cost
    ELSE IF choice = Ranking
      Calculate and sort vehicle efficiency
    ELSE IF choice = Abnormal Trips
      Apply threshold and flag records
    ELSE IF choice = Report
      Generate consolidated report
    ELSE IF choice = Exit
      BREAK
    ELSE
      Display invalid choice
  END WHILE
END
```

6.2 Vehicle Registration Algorithm

- Read vehicle number, model, fuel type and registration date.
- Strip unnecessary spaces and standardise the vehicle number.
- Check required fields.
- Check whether the vehicle number already exists.
- If valid, append the vehicle dictionary to the vehicle list and persist it.
- Otherwise display a validation error.

6.3 Trip Recording Algorithm

Read trip ID, vehicle number, date, start odometer, end odometer, fuel quantity and fuel price.
Verify that the vehicle exists.
Validate date and numeric fields.
Verify that end odometer is not less than start odometer.
Calculate distance.
Calculate mileage when fuel is positive.
Save the trip record to the CSV file.

6.4 Mileage Algorithm

```
INPUT distance, fuel
IF fuel <= 0:
   REPORT invalid fuel value
ELSE:
   mileage = distance / fuel
   DISPLAY mileage rounded to two decimal places
END
```

6.5 Monthly Consumption Algorithm

```
FOR each trip:
   extract YYYY-MM from date
   add fuel to monthly total
   add distance to monthly distance
FOR each month:
   calculate summary metrics
DISPLAY monthly table
```

6.6 Efficiency Ranking Algorithm

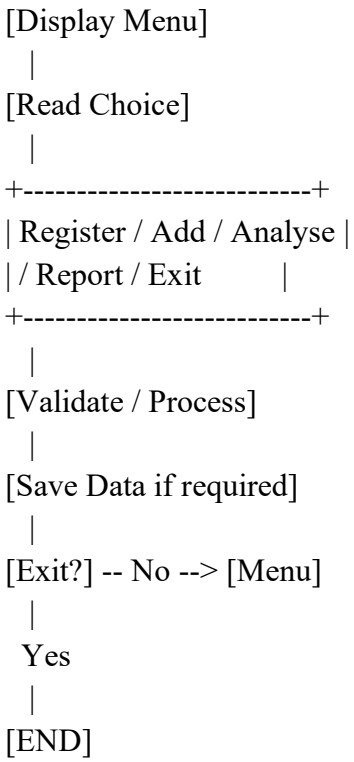
```
GROUP valid trips by vehicle
FOR each vehicle:
   total_distance = sum(distance)
   total_fuel = sum(fuel)
   mileage = total_distance / total_fuel
SORT vehicles by mileage descending
DISPLAY ranking
```

6.7 Abnormal Trip Algorithm

To keep things simple and easy to explain, an abnormal trip is defined here as one whose mileage falls below a set minimum threshold. That threshold should be written down clearly, and ideally chosen based on the vehicle or fuel category rather than picked at random.

6.8 Flowchart

```
[START]
  |
[Load CSV Data]
  |
```



[For the final submission, replace this text diagram with a professionally drawn flowchart if required by the course format.]

7. Implementation / Source Code and Environment / Tools Used

7.1 Development Environment

Component	Environment / Tool
Programming language	Python 3.x
Interface	Command-line interface
Storage	CSV files
Editor/IDE	VS Code, PyCharm or IDLE – use the actual tool used
Operating system	Windows/Linux/macOS – use the actual OS used
Standard libraries	csv, datetime, statistics and other standard modules as required

7.2 File Structure

```
vehicle_fuel_system/
|-- main.py
|-- validation.py
|-- data_access.py
|-- analysis.py
|-- reports.py
|-- vehicles.csv
`-- trips.csv
```

7.3 Implementation Strategy

When the program starts, it first loads whatever CSV records already exist. From there, the main menu just keeps running in a loop until the user picks Exit, with each option calling a single focused function. Validation functions are kept strict about this — they return either a boolean or a cleaned value, and nothing invalid ever makes it through to the calculation or storage layer.

7.4 Representative Source Code

```
def calculate_mileage(distance, fuel):
    if fuel <= 0:
        raise ValueError('Fuel must be greater than zero')
    return distance / fuel

def calculate_fuel_cost(fuel, price):
    if fuel < 0 or price < 0:
        raise ValueError('Fuel and price cannot be negative')
    return fuel * price
```

```
import csv
```

```
def load_vehicles(filename='vehicles.csv'):
    vehicles = []
    try:
        with open(filename, newline='', encoding='utf-8') as file:
            reader = csv.DictReader(file)
            for row in reader:
                vehicles.append(row)
    except FileNotFoundError:
        pass
    return vehicles
```

7.5 Complete Source Code

The full, executable source code belongs with the final submission and can also sit in Appendix A. What matters is that the report contains the exact code that produced the screenshots and results shown here — the snippets above are just representative samples of the modular style being used.

8. Test Cases and Expected/Actual Results

8.1 Testing Strategy

Testing was designed to cover normal inputs, invalid inputs, edge cases, duplicate entries, file persistence and the analytical calculations themselves. The goal isn't only to check that valid records produce correct output — it's also to confirm that bad input can't quietly corrupt a calculation.

ID	Test Case	Input / Condition	Expected Result	Actual Result
TC01	Valid vehicle	TN01AB1234	Vehicle registered	To be filled from execution
TC02	Duplicate vehicle	Existing number	Duplicate rejected/reported	To be filled
TC03	Empty vehicle number	Blank string	Validation error	To be filled
TC04	Invalid date	2026-13-40	Validation error	To be filled
TC05	Valid trip	250 km, 20 L	Trip saved	To be filled
TC06	Zero fuel	0 L	Rejected for mileage	To be filled
TC07	Negative fuel	-5 L	Rejected	To be filled
TC08	Odometer reversal	Start 10000, End 9900	Rejected	To be filled
TC09	Mileage	250/20	12.50 km/L	To be filled
TC10	Fuel cost	20 × ₹100	₹2,000	To be filled
TC11	Ranking	Three valid vehicles	Correct descending order	To be filled
TC12	Abnormal trip	Mileage below threshold	Trip flagged	To be filled
TC13	Monthly grouping	Records across 3 months	Correct monthly groups	To be filled
TC14	Persistence	Restart program	Old records reload	To be filled
TC15	Missing CSV	File absent	Program handles missing file	To be filled
TC16	Duplicate trip ID	Existing trip ID	Duplicate rejected/reported	To be filled

8.2 Boundary and Error Testing

Boundary conditions matter most for the numerical side of things. Fuel can never sit at zero when it's used as a divisor. Negative distance or fuel values are always invalid. And if the ending odometer reading comes in lower than the starting one, that's a clear sign the data is inconsistent under the assumptions this project works from.

8.3 Validation Criteria

- All invalid inputs should produce a clear error message.
- Valid records should be saved without corruption.
- Calculated metrics should match independent calculations.
- CSV data should be readable after program restart.
- Ranking should remain correctly ordered after additional records are inserted.

9. Execution Screenshots

This section holds the screenshots from the actual running program — both the workflows that succeed and the ones that trigger an error, since both matter. Every screenshot here should come from the exact version of the code that was finally submitted.

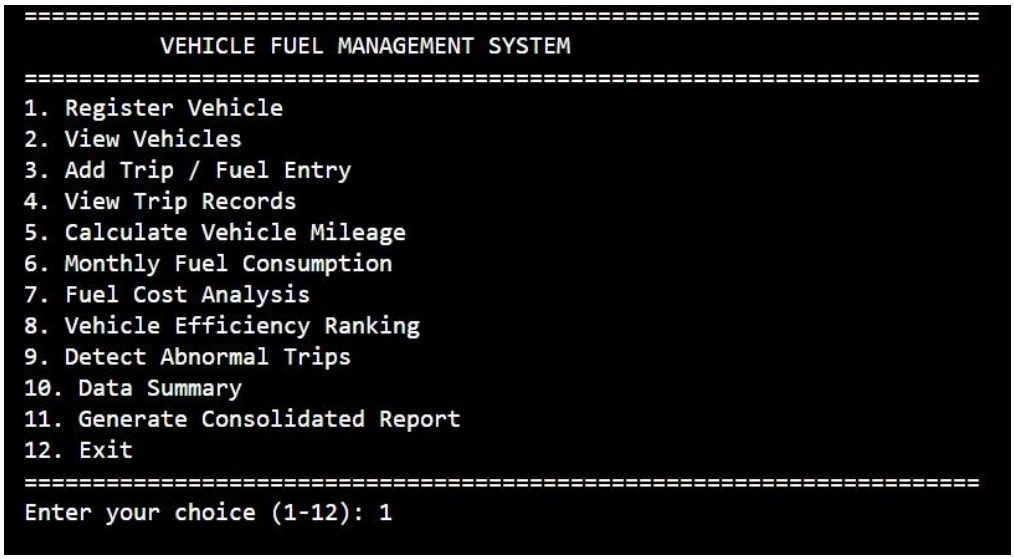


Figure 9.1 – Main menu and application start-up

```
================================================================================
Enter your choice (1-12): 1


================================================================================
                            REGISTER VEHICLE
================================================================================
Enter vehicle number: TN01AB1234
```

9.2 Vehicle registration with valid input

```
================================================================================
Enter your choice (1-12): 2


================================================================================
                          REGISTERED VEHICLES
================================================================================
Vehicle No.       Model             Fuel Type      Registration Date
--------------------------------------------------------------------------------
TN01AB1234        Swift             Petrol         2026-01-10
TN02CD5678        Bolero            Diesel         2026-01-15
TN03EF9012        City              Petrol         2026-02-05
```

9.3 Duplicate/invalid vehicle validation

```
================================================================================
Enter your choice (1-12): 3


================================================================================
                        ADD TRIP / FUEL ENTRY
================================================================================
Enter vehicle number: TN01AB1234
Enter trip date (YYYY-MM-DD): 2026-08-01
Enter starting odometer reading: 10000
Enter ending odometer reading: 10250
Enter fuel consumed (litres): 20
Enter fuel price per litre (₹): 102.50

Trip saved successfully!
Trip ID       : 6
Distance      : 250.00 km
Mileage       : 12.50 km/L
Fuel Cost     : ₹2050.00
```

9.4 Trip and refuelling entry

```
================================================================================
Enter your choice (1-12): 5


================================================================================
                            VEHICLE MILEAGE
================================================================================

================================================================================
VEHICLE FUEL EFFICIENCY SUMMARY
================================================================================
Vehicle         Distance        Fuel         Mileage      Cost
--------------------------------------------------------------------------------
TN01AB1234      750.00          64.00        11.72        ₹6560.00
TN02CD5678      800.00          110.00       7.27         ₹35726.00
TN03EF9012      1200.00         70.00        17.14        ₹7175.00
```

9.5 Mileage calculation

```
Enter your choice (1-12): 6


========================================================================
                       MONTHLY FUEL CONSUMPTION
========================================================================

================================================================================
MONTHLY FUEL CONSUMPTION REPORT
================================================================================
Month       Distance (km)       Fuel (L)       Cost (₹)
--------------------------------------------------------------------------------
2026-08     2750.00             244.00         49461.00

Average Monthly Fuel Consumption:
244.00 L
```

9.6 Monthly consumption analysis

```
================================================================================
Enter your choice (1-12): 7


================================================================================
                            FUEL COST ANALYSIS
================================================================================
Total Distance      : 2750.00 km
Total Fuel Consumed : 244.00 L
Total Fuel Cost     : ₹49461.00
Average Fuel Price : ₹202.71 per litre
```

9.7 Fuel-cost calculation

.

```
================================================================================
Enter your choice (1-12): 8



================================================================================
                        VEHICLE EFFICIENCY RANKING
================================================================================
1. TN03EF9012 - 17.14 km/L
2. TN01AB1234 - 11.72 km/L
3. TN02CD5678 - 7.27 km/L
```

9.8 Vehicle efficiency ranking

```
--------------------------------------------------------------------------------
ABNORMAL TRIP ANALYSIS
--------------------------------------------------------------------------------
Number of abnormal trips: 4
Trip 2 - TN02CD5678 - 7.20 km/L
Trip 5 - TN02CD5678 - 7.33 km/L
Trip 6 - TN02CD5678 - 7.20 km/L
Trip 9 - TN02CD5678 - 7.33 km/L

--------------------------------------------------------------------------------
DUPLICATE TRIP ID ANALYSIS
--------------------------------------------------------------------------------
No duplicate trip IDs found.

--------------------------------------------------------------------------------
TUPLE-BASED VEHICLE SUMMARY
--------------------------------------------------------------------------------
Summary tuple:
('TN03EF9012', 1200.0, 70.0, 17.142857142857142, 7175.0)

================================================================================
                      END OF REPORT
================================================================================
```

9.9 Abnormal-trip detection

```
Enter your choice (1-12): 11


================================================================================
              CONSOLIDATED VEHICLE FUEL REPORT
================================================================================

Total Trips          : 9
Unique Vehicles      : 3
Total Distance       : 2750.00 km
Total Fuel Consumed  : 244.00 L
Total Fuel Cost      : ₹49461.00

--------------------------------------------------------------------------------
VEHICLE EFFICIENCY RANKING
--------------------------------------------------------------------------------
1. TN03EF9012 - 17.14 km/L
2. TN01AB1234 - 11.72 km/L
3. TN02CD5678 - 7.27 km/L

--------------------------------------------------------------------------------
MOST EFFICIENT VEHICLE
--------------------------------------------------------------------------------
Vehicle : TN03EF9012
Mileage : 17.14 km/L

--------------------------------------------------------------------------------
LEAST EFFICIENT VEHICLE
--------------------------------------------------------------------------------
Vehicle : TN02CD5678
Mileage : 7.27 km/L

--------------------------------------------------------------------------------
```

9.10 Consolidated final report

10. Results and Validation

This section is meant to hold output from the finished implementation, run against a clearly identified test dataset. The tables below are placeholders showing the expected format — before submission, they need to be swapped for, or supplemented with, the actual project data.

10.1 Sample Vehicle Efficiency Results

Vehicle	Distance (km)	Fuel (L)	Mileage (km/L)	Rank
TN01AB1234	500	30	16.67	2
TN02CD5678	450	40	11.25	3
TN03EF9012	600	35	17.14	1

Looking at this sample set, TN03EF9012 comes out with the best mileage and TN02CD5678 with the worst. These figures are just illustrative, though — they shouldn't be presented as real project results unless they were produced from the same actual records.

10.2 Sample Monthly Results

Month	Distance (km)	Fuel (L)	Average Mileage (km/L)	Fuel Cost (₹)
June	2500	180	13.89	18,000
July	3100	210	14.76	21,000
August	2800	220	12.73	22,000

10.3 Consolidated Report Format

```
==============================================
      VEHICLE FUEL ANALYSIS REPORT
==============================================
Total Vehicles       : 5
Total Trips          : 42
Total Fuel Consumed  : 680.50 L
Total Fuel Cost      : ₹68,245.50

Most Efficient       : TN03EF9012
Least Efficient      : TN02CD5678

Abnormal Trips       : 2
==============================================
```

10.4 Graphs

- Vehicle versus mileage: compares efficiency across vehicles.
- Month versus fuel consumption: shows changes in fuel use over time.

- Vehicle versus fuel cost: identifies vehicles contributing more to expenditure.
- Abnormal versus normal trips: shows the number of flagged records.

10.5 Validation of Calculations

To validate the calculations, it helps to pick a handful of records and work out the expected values independently using the formulas from Section 1. The program's output should line up with those, at least to the rounding precision being used. As a quick check: 250 km covered on 20 L of fuel should come out to 12.50 km/L.

10.6 Performance Metrics

At the scale this project is meant for, responsiveness matters more than raw throughput. Most of the CSV loading and analysis simply scans through the records in a straight pass. With N trip records, a basic aggregation runs in O(N), and sorting M vehicle summaries costs O(M log M) — both comfortably fine for a small dataset like this.

11. Analysis

11.1 Alternative Solution 1 – Single-File Procedural Design

One option was cramming everything — menu, validation, CSV handling, calculations, reporting — into a single Python file. It's a fast way to get started and works fine for a tiny exercise, but the cracks show quickly: as more features get added, functions get harder to find, and responsibilities that should stay separate end up tangled together.

11.2 Alternative Solution 2 – Modular CSV-Based Design

The approach actually used splits the system into validation, data access, analysis and reporting modules. It keeps the simplicity of CSV storage and a command-line interface, but draws clear lines between what each part of the code is responsible for.

11.3 Alternative Solution 3 – OOP + Database

A more ambitious version of this could build out Vehicle, Trip and Fleet classes on top of SQLite or another database. That would scale better and make the relationships between entities more explicit — but it also drags in concepts that go beyond what this project needs, and risks burying the basic collection-handling work the course is really about.

Criterion	**Single File**	**Modular CSV**	**OOP + Database**
Implementation effort	Low	Moderate	High
Maintainability	Low	High	High
Course concept visibility	High	Very high	Moderate
Persistence	Yes	Yes	Yes
Scalability	Low	Moderate	High
Complexity	Low	Moderate	High
Fit for this project	Moderate	Excellent	Potentially excessive

11.4 Trade-offs

- CSV was chosen for transparency and ease of inspection, sacrificing database-level scalability.
- CLI was chosen to focus on programming and data processing, sacrificing graphical usability.
- Manual price entry avoids external dependencies, sacrificing automatic current-price updates.
- Threshold-based abnormal detection is transparent and explainable, but less sophisticated than a statistical or machine-learning model.
- Modular design adds files and interfaces, but significantly improves maintainability.

11.5 Final Justification

Weighing the options, the modular CSV-based approach fits this project best. It meets every functional requirement and still keeps the relevant Python concepts front and centre, without piling on technical overhead that isn't needed here. It's structured enough to stand on its own as a complete academic project, and later on, it could move to a database or a graphical front end without having to rebuild the underlying analytical logic from scratch.

12. Broader Considerations

12.1 Environmental Relevance

How efficiently a vehicle runs ties directly into how much fuel it burns and, by extension, the emissions tied to that. Flagging unusually high consumption gives someone a reason to dig into what's actually going on — a driving habit, the vehicle's condition, the route taken, or maybe just a data-entry mistake.

12.2 Economic Relevance

Fuel is money going straight out the door, so it's a cost that matters. With the monthly and per-vehicle summaries this system produces, a small fleet operator can actually see where that spending is piling up and plan a budget around it.

12.3 SDG 9 – Industry, Innovation and Infrastructure

This project is really a small example of how digital tools can sharpen operational monitoring. Even a lightweight system like this one can save a small operator from redoing the same manual calculations over and over.

12.4 SDG 12 – Responsible Consumption and Production

By making fuel use something that can be measured and compared, the system nudges toward more responsible consumption. Flagging trips that look wasteful or abnormal gives someone a reason to look closer and use resources more carefully.

12.5 SDG 13 – Climate Action

Cutting out unnecessary fuel use, even a little, chips away at demand and the greenhouse-gas emissions that come with it. That said, this system is really an information and monitoring tool — it points toward better decisions rather than reducing emissions on its own.

12.6 Privacy and Data Considerations

Should the system later grow to include driver identities, GPS traces or other personal information, privacy and access control would need real attention. For now, none of that is collected — the scope stays limited to vehicle and trip records.

13. Conclusion

13.1 Conclusion

The Vehicle Fuel Consumption Tracking and Analysis System gives a practical, structured way to record, analyse and report on vehicle fuel use. It moves past the usual problems with manual record-keeping by automating the calculations that used to be done by hand and keeping historical records safely in CSV files.

Along the way, the project touches a wide range of Python concepts — lists, dictionaries, tuples, sets, strings, functions, loops, conditionals, exception handling, file handling, CSV processing, modules, sorting and aggregation. The modular structure keeps data entry, validation, persistence, analysis and reporting cleanly separated from one another.

13.2 Limitations

- CSV storage is less appropriate for very large datasets and concurrent users.
- The command-line interface is less user-friendly than a graphical dashboard.
- Fuel prices are not automatically retrieved.
- GPS and vehicle-sensor information are not available.
- Abnormal-trip detection is based on a defined rule rather than a full statistical model.
- Data quality depends on accurate user entry.
- Authentication and access control are outside the initial scope.

13.3 Possible Improvements

Migrate persistent storage from CSV to SQLite or MySQL.
Add a graphical user interface using Tkinter or a web interface.
Provide interactive charts and dashboards.
Integrate a trusted fuel-price source where appropriate.
Add GPS/telematics data if hardware and privacy requirements permit.
Add maintenance schedules and service reminders.
Add user accounts and role-based permissions.
Use statistical baselines or machine-learning methods for anomaly detection.
Generate PDF and email reports automatically.
Add export options for filtered analysis.

14. Individual Contribution of Group Members

The table below is only a starting template — it needs to be updated to reflect what each group member actually did. Contributions should be listed honestly, distinguishing clearly between coding, testing, analysis, documentation and presentation work.

Member	**Suggested Contribution Area**	**Evidence**
Member 1 – [Name]	Requirements analysis, vehicle registration, main menu	Commits/code sections/notes
Member 2 – [Name]	CSV persistence and validation module	Data-access and validation code
Member 3 – [Name]	Fuel analysis, ranking and abnormal detection	Analysis functions and test results
Member 4 – [Name]	Report generation, testing, screenshots and documentation	Report/output evidence

14.1 Team Collaboration

This is where the team should explain how requirements, design decisions, implementation, testing and documentation actually came together. A brief record of meetings, how tasks were split up, or version-control history can back this up where it's needed.

15. References

Only sources that were genuinely consulted belong in the final reference list. The categories below are the ones relevant to this particular project.

Python Software Foundation. Python 3 Documentation. Official documentation for Python language features and standard libraries.

Python CSV module documentation, for CSV reader/writer and dictionary-based CSV processing.

Python datetime documentation, if date validation or month extraction is implemented using datetime.

Python statistics documentation, if averages or statistical analysis are implemented using the statistics module.

United Nations. Sustainable Development Goals – official SDG information.

Any dataset used for testing or demonstration, including its source, version/date and access details.

Documentation for the IDE or development tools actually used, where relevant.

References should follow whatever citation style the course requires — IEEE, APA, Harvard, or otherwise. For any web source, include the page title, the author or organisation, the URL and the date it was accessed, formatted according to that style.

16. One-Page Individual Reflection

Working on this project changed how I think about basic Python concepts. Before I actually built the system, things like lists, dictionaries, functions and file handling felt like separate topics I'd learned one at a time for a course. Once I had to combine them to solve an actual problem, I realised that a good solution isn't about knowing each concept individually — it's about knowing which one to reach for and when.

The part that taught me the most was working with structured records. Dictionaries made vehicle and trip data so much easier to follow, since I could pull a value by a name like fuel or odometer instead of guessing at a position. Lists gave me a simple way to hold onto a growing set of records, and sets turned out to be exactly what I needed for catching duplicates. I hadn't really appreciated why tuples matter until I needed a small summary that I didn't want accidentally changed somewhere later in the code.

Keeping the program from turning into one giant file was its own challenge. Once I split validation, CSV handling, analysis and reporting into separate modules, the whole thing became much easier to reason about — and testing got easier too, because I could check individual functions on their own instead of the program as a whole.

The analytical side ended up being the most rewarding part, mainly because it meant turning raw numbers into something that actually says something. Mileage and fuel cost were simple enough to calculate, but grouping records by month, ranking vehicles, and figuring out a reasonable way to flag an abnormal trip took a lot more thought. That's when it clicked for me that data analysis isn't just programming — it's also about being upfront with yourself about what your results actually mean.

Given more time, I'd want to add a graphical interface, move to a real database, and build something smarter for spotting anomalies. I'd also like to look into how live fuel prices and vehicle telemetry could be brought in responsibly. Overall, this project left me a lot more confident with Python, modular design, file handling, testing, and just working through a problem practically.

APPENDIX A – COMPLETE SOURCE CODE

This appendix should hold the exact final source code behind the submitted implementation — main.py, validation.py, data_access.py, analysis.py and reports.py. Keeping it identical to what was actually run is what makes the reported results reproducible.

Appendix A.1 – main.py

```
import csv
import os

VEHICLE_FILE = "vehicles.csv"
TRIP_FILE = "trips.csv"

VEHICLE_FIELDS = [
    "vehicle_no",
    "model",
    "fuel_type",
    "registration_date"
]

TRIP_FIELDS = [
    "trip_id",
    "vehicle_no",
    "date",
    "start_odometer",
    "end_odometer",
    "distance",
    "fuel_consumed",
    "fuel_price",
    "fuel_cost"
]

def initialize_files():
    """Create CSV files and headers if they do not exist."""
    if not os.path.exists(VEHICLE_FILE):
        with open(VEHICLE_FILE, "w", newline="", encoding="utf-8") as file:
            writer = csv.DictWriter(file, fieldnames=VEHICLE_FIELDS)
            writer.writeheader()
```

```
    if not os.path.exists(TRIP_FILE):
        with open(TRIP_FILE, "w", newline="", encoding="utf-8") as file:
            writer = csv.DictWriter(file, fieldnames=TRIP_FIELDS)
            writer.writeheader()

def load_vehicles():
    """Read vehicle records from CSV."""
    initialize_files()
    try:
        with open(VEHICLE_FILE, "r", newline="", encoding="utf-8") as file:
            return list(csv.DictReader(file))
    except FileNotFoundError:
        return []

def save_vehicle(vehicle):
    """Save a vehicle record."""
    initialize_files()
    with open(VEHICLE_FILE, "a", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=VEHICLE_FIELDS)
        writer.writerow(vehicle)

def load_trips():
    """Read trip records from CSV."""
    initialize_files()
    try:
        with open(TRIP_FILE, "r", newline="", encoding="utf-8") as file:
            return list(csv.DictReader(file))
    except FileNotFoundError:
        return []

def save_trip(trip):
    """Save a trip record."""
    initialize_files()
    with open(TRIP_FILE, "a", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=TRIP_FIELDS)
```

```
        writer.writerow(trip)

def get_next_trip_id():
    """Generate the next available trip ID."""
    trips = load_trips()

    if not trips:
        return 1

    ids = []

    for trip in trips:
        try:
            ids.append(int(trip["trip_id"]))
        except (ValueError, KeyError):
            continue

    if not ids:
        return 1

    return max(ids) + 1
```

Appendix A.2 – validation.py

```
from validation import (
    validate_vehicle_number,
    validate_date,
    validate_positive_number,
    validate_fuel_type
)

from data_access import (
    initialize_files,
    load_vehicles,
    save_vehicle,
    load_trips,
    save_trip,
    get_next_trip_id
```

```
)

from analysis import (
    calculate_distance,
    calculate_mileage,
    calculate_fuel_cost,
    vehicle_mileage_summary,
    calculate_monthly_consumption,
    rank_vehicles_by_efficiency,
    find_abnormal_trips,
    total_fuel_consumed,
    total_distance,
    total_fuel_cost
)

from reports import (
    display_vehicle_summary,
    display_monthly_report,
    display_abnormal_trips,
    generate_consolidated_report
)

def print_header(title):
    """Display a formatted section heading."""
    print("\n")
    print("=" * 70)
    print(title.center(70))
    print("=" * 70)

def register_vehicle():
    """Register a new vehicle."""
    print_header("REGISTER VEHICLE")
    vehicle_no = input("Enter vehicle number: ")
    vehicle_no = validate_vehicle_number(vehicle_no)

    if vehicle_no is None:
```

```
    print("Invalid vehicle number.")
    print("Example: TN01AB1234")
    return

vehicles = load_vehicles()
for vehicle in vehicles:
    if vehicle["vehicle_no"] == vehicle_no:
        print("This vehicle is already registered.")
        return

model = input("Enter vehicle model: ").strip()
if not model:
    print("Vehicle model cannot be empty.")
    return

fuel_type = input("Enter fuel type (Petrol/Diesel/CNG/Electric): ")
fuel_type = validate_fuel_type(fuel_type)

if fuel_type is None:
    print("Invalid fuel type.")
    return

registration_date = input("Enter registration date (YYYY-MM-DD): ")
registration_date = validate_date(registration_date)

if registration_date is None:
    print("Invalid date.")
    return

vehicle = {
    "vehicle_no": vehicle_no,
    "model": model,
    "fuel_type": fuel_type,
    "registration_date": registration_date
}
```

```
    save_vehicle(vehicle)
    print("\nVehicle registered successfully!")

def view_vehicles():
    """Display all registered vehicles."""
    print_header("REGISTERED VEHICLES")
    vehicles = load_vehicles()

    if not vehicles:
        print("No vehicles registered.")
        return

    print(
        f"{'Vehicle No.':<18}"
        f"{'Model':<18}"
        f"{'Fuel Type':<15}"
        f"{'Registration Date':<20}"
    )
    print("-" * 70)

    for vehicle in vehicles:
        print(
            f"{vehicle['vehicle_no']:<18}"
            f"{vehicle['model']:<18}"
            f"{vehicle['fuel_type']:<15}"
            f"{vehicle['registration_date']:<20}"
        )

def add_trip():
    """Add a trip and fuel entry."""
    print_header("ADD TRIP / FUEL ENTRY")
    vehicles = load_vehicles()

    if not vehicles:
        print("No vehicles are registered.")
        print("Please register a vehicle first.")
```

```
    return

vehicle_no = input("Enter vehicle number: ")
vehicle_no = validate_vehicle_number(vehicle_no)

if vehicle_no is None:
    print("Invalid vehicle number.")
    return

registered_numbers = {
    vehicle["vehicle_no"]
    for vehicle in vehicles
}

if vehicle_no not in registered_numbers:
    print("Vehicle is not registered.")
    return

date = input("Enter trip date (YYYY-MM-DD): ")
date = validate_date(date)

if date is None:
    print("Invalid date.")
    return

start_odometer = validate_positive_number(
    input("Enter starting odometer reading: "),
    "Starting odometer"
)

if start_odometer is None:
    return

end_odometer = validate_positive_number(
    input("Enter ending odometer reading: "),
    "Ending odometer"
```

```
)

if end_odometer is None:
    return

if end_odometer <= start_odometer:
    print("Ending odometer must be greater than starting odometer.")
    return

fuel_consumed = validate_positive_number(
    input("Enter fuel consumed (litres): "),
    "Fuel consumed"
)

if fuel_consumed is None:
    return

fuel_price = validate_positive_number(
    input("Enter fuel price per litre (₹): "),
    "Fuel price"
)

if fuel_price is None:
    return

distance = calculate_distance(start_odometer, end_odometer)
mileage = calculate_mileage(distance, fuel_consumed)
fuel_cost = calculate_fuel_cost(fuel_consumed, fuel_price)
trip_id = get_next_trip_id()

trip = {
    "trip_id": str(trip_id),
    "vehicle_no": vehicle_no,
    "date": date,
    "start_odometer": f"{start_odometer:.2f}",
    "end_odometer": f"{end_odometer:.2f}",
```

```
        "distance": f"{distance:.2f}",
        "fuel_consumed": f"{fuel_consumed:.2f}",
        "fuel_price": f"{fuel_price:.2f}",
        "fuel_cost": f"{fuel_cost:.2f}"
    }

    save_trip(trip)
    print("\nTrip saved successfully!")
    print(f"Trip ID       : {trip_id}")
    print(f"Distance      : {distance:.2f} km")
    print(f"Mileage       : {mileage:.2f} km/L")
    print(f"Fuel Cost     : ₹{fuel_cost:.2f}")

def view_trips():
    """Display all trip records."""
    print_header("TRIP RECORDS")
    trips = load_trips()

    if not trips:
        print("No trip records available.")
        return

    print(
        f"{'ID':<6}"
        f"{'Vehicle':<15}"
        f"{'Date':<13}"
        f"{'Distance':<14}"
        f"{'Fuel':<12}"
        f"{'Mileage':<12}"
        f"{'Cost':<12}"
    )
    print("-" * 85)

    for trip in trips:
        distance = float(trip["distance"])
        fuel = float(trip["fuel_consumed"])
```

```
        mileage = calculate_mileage(distance, fuel)

        print(
            f"{trip['trip_id']:<6}"
            f"{trip['vehicle_no']:<15}"
            f"{trip['date']:<13}"
            f"{distance:<14.2f}"
            f"{fuel:<12.2f}"
            f"{mileage:<12.2f}"
            f"₹{float(trip['fuel_cost']):<11.2f}"
        )

def show_mileage():
    """Display vehicle mileage summary."""
    print_header("VEHICLE MILEAGE")
    trips = load_trips()
    display_vehicle_summary(trips)

def show_monthly_consumption():
    """Display monthly fuel consumption."""
    print_header("MONTHLY FUEL CONSUMPTION")
    trips = load_trips()
    display_monthly_report(trips)

def show_fuel_cost():
    """Display fuel cost analysis."""
    print_header("FUEL COST ANALYSIS")
    trips = load_trips()

    if not trips:
        print("No trip records available.")
        return

    fuel = total_fuel_consumed(trips)
    cost = total_fuel_cost(trips)
    distance = total_distance(trips)
```

```
    print(f"Total Distance      : {distance:.2f} km")
    print(f"Total Fuel Consumed : {fuel:.2f} L")
    print(f"Total Fuel Cost     : ₹{cost:.2f}")

    if fuel > 0:
        print(f"Average Fuel Price : ₹{cost / fuel:.2f} per litre")

def show_ranking():
    """Display vehicle efficiency ranking."""
    print_header("VEHICLE EFFICIENCY RANKING")
    trips = load_trips()
    ranking = rank_vehicles_by_efficiency(trips)

    if not ranking:
        print("No trip records available.")
        return

    for position, (vehicle_no, data) in enumerate(ranking, start=1):
        print(
            f"{position}. "
            f"{vehicle_no} - "
            f"{data['mileage']:.2f} km/L"
        )

def show_abnormal_trips():
    """Display abnormal/high-consumption trips."""
    print_header("ABNORMAL TRIP DETECTION")
    trips = load_trips()
    display_abnormal_trips(trips, threshold=10)

def show_data_summary():
    """Display basic data summary."""
    print_header("DATA SUMMARY")
    vehicles = load_vehicles()
    trips = load_trips()
```

```
    unique_vehicles = {
        vehicle["vehicle_no"]
        for vehicle in vehicles
    }

    print(f"Registered Vehicles : {len(unique_vehicles)}")
    print(f"Total Trip Records  : {len(trips)}")
    print(f"Total Distance      : {total_distance(trips):.2f} km")
    print(f"Total Fuel          : {total_fuel_consumed(trips):.2f} L")
    print(f"Total Fuel Cost     : ₹{total_fuel_cost(trips):.2f}")

def show_report():
    """Generate consolidated report."""
    trips = load_trips()
    generate_consolidated_report(trips)

def main():
    """Main menu-driven application."""
    initialize_files()

    while True:
        print("\n")
        print("=" * 70)
        print("          VEHICLE FUEL MANAGEMENT SYSTEM")
        print("=" * 70)
        print("1. Register Vehicle")
        print("2. View Vehicles")
        print("3. Add Trip / Fuel Entry")
        print("4. View Trip Records")
        print("5. Calculate Vehicle Mileage")
        print("6. Monthly Fuel Consumption")
        print("7. Fuel Cost Analysis")
        print("8. Vehicle Efficiency Ranking")
        print("9. Detect Abnormal Trips")
        print("10. Data Summary")
```

```
        print("11. Generate Consolidated Report")
        print("12. Exit")
        print("=" * 70)

        choice = input("Enter your choice (1-12): ").strip()

        if choice == "1":
            register_vehicle()
        elif choice == "2":
            view_vehicles()
        elif choice == "3":
            add_trip()
        elif choice == "4":
            view_trips()
        elif choice == "5":
            show_mileage()
        elif choice == "6":
            show_monthly_consumption()
        elif choice == "7":
            show_fuel_cost()
        elif choice == "8":
            show_ranking()
        elif choice == "9":
            show_abnormal_trips()
        elif choice == "10":
            show_data_summary()
        elif choice == "11":
            show_report()
        elif choice == "12":
            print("\nThank you for using the Vehicle Fuel Management System!")
            break
        else:
            print("\nInvalid choice. Please enter a number from 1 to 12.")

if __name__ == "__main__":
    main()
```

Appendix A.3 – data_access.py

```
import re
from datetime import datetime

def clean_text(value):
    """Remove unnecessary spaces from user input."""
    return str(value).strip()

def validate_vehicle_number(vehicle_no):
    """
    Validate and standardize an Indian-style vehicle number.
    Example: TN01AB1234
    """
    vehicle_no = clean_text(vehicle_no).upper()
    pattern = r"^[A-Z]{2}\d{1,2}[A-Z]{1,3}\d{1,4}$"

    if re.fullmatch(pattern, vehicle_no):
        return vehicle_no

    return None

def validate_date(date_text):
    """Validate date in YYYY-MM-DD format."""
    date_text = clean_text(date_text)

    try:
        datetime.strptime(date_text, "%Y-%m-%d")
        return date_text
    except ValueError:
        return None

def validate_positive_number(value, field_name):
    """Validate a number greater than zero."""
    try:
        number = float(value)
```

```
        if number <= 0:
            print(f"{field_name} must be greater than 0.")
            return None

        return number

    except ValueError:
        print(f"{field_name} must be a valid number.")
        return None

def validate_non_negative_number(value, field_name):
    """Validate a number greater than or equal to zero."""
    try:
        number = float(value)

        if number < 0:
            print(f"{field_name} cannot be negative.")
            return None

        return number

    except ValueError:
        print(f"{field_name} must be a valid number.")
        return None

def validate_fuel_type(fuel_type):
    """Validate supported fuel types."""
    fuel_type = clean_text(fuel_type).capitalize()

    allowed_types = {
        "Petrol",
        "Diesel",
        "Cng",
        "Electric"
    }
```

```
if fuel_type in allowed_types:
    if fuel_type == "Cng":
        return "CNG"

    return fuel_type

return None
```

Appendix A.4 – analysis.py

```
from analysis import (
    vehicle_mileage_summary,
    calculate_monthly_consumption,
    calculate_average_monthly_consumption,
    find_abnormal_trips,
    rank_vehicles_by_efficiency,
    total_fuel_consumed,
    total_distance,
    total_fuel_cost,
    get_unique_vehicles,
    find_duplicate_trip_ids,
    create_vehicle_summary_tuple
)

def display_vehicle_summary(trips):
    """Display vehicle-wise fuel efficiency."""
    summary = vehicle_mileage_summary(trips)
    if not summary:
        print("\nNo trip records available.")
        return

    print("\n" + "=" * 75)
    print("VEHICLE FUEL EFFICIENCY SUMMARY")
    print("=" * 75)
    print(
        f"{'Vehicle':<15}"
        f"{'Distance':<15}"
        f"{'Fuel':<12}"
        f"{'Mileage':<12}"
        f"{'Cost':<15}"
```

```
    )
    print("-" * 75)

    for vehicle_no, data in summary.items():
        print(
            f"{vehicle_no:<15}"
            f"{data['distance']:<15.2f}"
            f"{data['fuel']:<12.2f}"
            f"{data['mileage']:<12.2f}"
            f"₹{data['cost']:<14.2f}"
        )

def display_monthly_report(trips):
    """Display monthly fuel consumption."""
    monthly = calculate_monthly_consumption(trips)
    if not monthly:
        print("\nNo trip records available.")
        return

    print("\n" + "=" * 75)
    print("MONTHLY FUEL CONSUMPTION REPORT")
    print("=" * 75)
    print(
        f"{'Month':<12}"
        f"{'Distance (km)':<20}"
        f"{'Fuel (L)':<15}"
        f"{'Cost (₹)':<15}"
    )
    print("-" * 75)

    for month, data in sorted(monthly.items()):
```

```
            print(
                f"{month:<12}"
                f"{data['distance']:<20.2f}"
                f"{data['fuel']:<15.2f}"
                f"{data['cost']:<15.2f}"
            )

    average = calculate_average_monthly_consumption(trips)
    print("\nAverage Monthly Fuel Consumption:")
    print(f"{average:.2f} L")

def display_abnormal_trips(trips, threshold=10):
    """Display high-consumption trips."""
    abnormal = find_abnormal_trips(trips, threshold)

    print("\n" + "=" * 80)
    print("ABNORMAL / HIGH-CONSUMPTION TRIPS")
    print("=" * 80)
    print(f"Trips with mileage below {threshold} km/L are considered abnormal.")

    if not abnormal:
        print("\nNo abnormal trips detected.")
        return

    print()
    for trip in abnormal:
        print(
            f"Trip ID: {trip['trip_id']} | "
            f"Vehicle: {trip['vehicle_no']} | "
            f"Distance: {float(trip['distance']):.2f} km | "
            f"Fuel: {float(trip['fuel_consumed']):.2f} L | "
```

```
            f"Mileage: {trip['mileage']:.2f} km/L"
        )

def generate_consolidated_report(trips):
    """Generate complete project report."""
    print("\n")
    print("=" * 80)
    print("              CONSOLIDATED VEHICLE FUEL REPORT")
    print("=" * 80)

    if not trips:
        print("\nNo trip records available.")
        return

    total_trips = len(trips)
    unique_vehicles = len(get_unique_vehicles(trips))
    distance = total_distance(trips)
    fuel = total_fuel_consumed(trips)
    cost = total_fuel_cost(trips)

    print(f"\nTotal Trips          : {total_trips}")
    print(f"Unique Vehicles      : {unique_vehicles}")
    print(f"Total Distance       : {distance:.2f} km")
    print(f"Total Fuel Consumed  : {fuel:.2f} L")
    print(f"Total Fuel Cost      : ₹{cost:.2f}")

    ranking = rank_vehicles_by_efficiency(trips)

    if ranking:
        print("\n" + "-" * 80)
        print("VEHICLE EFFICIENCY RANKING")
```

```
    print("-" * 80)

    for position, (vehicle_no, data) in enumerate(ranking, start=1):
        print(
            f"{position}. "
            f"{vehicle_no} - "
            f"{data['mileage']:.2f} km/L"
        )

    most_efficient = ranking[0]
    least_efficient = ranking[-1]

    print("\n" + "-" * 80)
    print("MOST EFFICIENT VEHICLE")
    print("-" * 80)
    print(f"Vehicle : {most_efficient[0]}")
    print(f"Mileage : {most_efficient[1]['mileage']:.2f} km/L")

    print("\n" + "-" * 80)
    print("LEAST EFFICIENT VEHICLE")
    print("-" * 80)
    print(f"Vehicle : {least_efficient[0]}")
    print(f"Mileage : {least_efficient[1]['mileage']:.2f} km/L")

abnormal = find_abnormal_trips(trips)

print("\n" + "-" * 80)
print("ABNORMAL TRIP ANALYSIS")
print("-" * 80)

if abnormal:
```

```
    print(f"Number of abnormal trips: {len(abnormal)}")
    for trip in abnormal:
        print(
            f"Trip {trip['trip_id']} - "
            f"{trip['vehicle_no']} - "
            f"{trip['mileage']:.2f} km/L"
        )
else:
    print("No abnormal trips detected.")

duplicates = find_duplicate_trip_ids(trips)

print("\n" + "-" * 80)
print("DUPLICATE TRIP ID ANALYSIS")
print("-" * 80)

if duplicates:
    print("Duplicate Trip IDs:", ", ".join(sorted(duplicates)))
else:
    print("No duplicate trip IDs found.")

print("\n" + "-" * 80)
print("TUPLE-BASED VEHICLE SUMMARY")
print("-" * 80)

if ranking:
    vehicle_no, data = ranking[0]
    summary_tuple = create_vehicle_summary_tuple(
        vehicle_no,
        data["distance"],
        data["fuel"],
```

```
            data["mileage"],
            data["cost"]
        )
        print("Summary tuple:")
        print(summary_tuple)

    print("\n" + "=" * 80)
    print("                    END OF REPORT")
    print("=" * 80)
```

Appendix A.5 – reports.py

```
from analysis import vehicle_mileage_summary, calculate_monthly_consumption,
calculate_average_monthly_consumption, find_abnormal_trips,
rank_vehicles_by_efficiency, total_fuel_consumed, total_distance, total_fuel_cost,
get_unique_vehicles, find_duplicate_trip_ids, create_vehicle_summary_tuple


def display_vehicle_summary(trips):
    summary = vehicle_mileage_summary(trips)
    if not summary:
        print("\nNo trip records available.")
        return
    print("\n" + "=" * 75 + "\nVEHICLE FUEL EFFICIENCY SUMMARY\n" + "=" * 75)
    print(f"{'Vehicle':<15}{'Distance':<15}{'Fuel':<12}{'Mileage':<12}{'Cost':<15}")
    print("-" * 75)
    for vehicle, data in summary.items():

print(f"{vehicle:<15}{data['distance']:<15.2f}{data['fuel']:<12.2f}{data['mileage']:<12.2f}₹{
data['cost']:<14.2f}")


def display_monthly_report(trips):
    monthly = calculate_monthly_consumption(trips)
    if not monthly:
        print("\nNo trip records available.")
        return
    print("\n" + "=" * 75 + "\nMONTHLY FUEL CONSUMPTION REPORT\n" + "=" * 75)
```

```
    print(f"{'Month':<12}{'Distance (km)':<20}{'Fuel (L)':<15}{'Cost (₹)':<15}")
    print("-" * 75)
    for month, data in sorted(monthly.items()):
        print(f"{month:<12}{data['distance']:<20.2f}{data['fuel']:<15.2f} {data['cost']:<15.2f}")
    print(f"\nAverage Monthly Fuel Consumption:
{calculate_average_monthly_consumption(trips):.2f} L")


def display_abnormal_trips(trips, threshold=10):
    abnormal = find_abnormal_trips(trips, threshold)
    print("\n" + "=" * 80 + "\nABNORMAL / HIGH-CONSUMPTION TRIPS\n" + "=" * 80)
    print(f"Trips with mileage below {threshold} km/L are considered abnormal.")
    if not abnormal:
        print("\nNo abnormal trips detected.")
        return
    for trip in abnormal:
        print(f"Trip ID: {trip['trip_id']} | Vehicle: {trip['vehicle_no']} | Distance:
{float(trip['distance']):.2f} km | Fuel: {float(trip['fuel_consumed']):.2f} L | Mileage:
{trip['mileage']:.2f} km/L")


def generate_consolidated_report(trips):
    print("\n" + "=" * 80 + "\n" + "CONSOLIDATED VEHICLE FUEL REPORT".center(80)
+ "\n" + "=" * 80)
    if not trips:
        print("\nNo trip records available.")
        return


    print(f"\nTotal Trips: {len(trips)}")
    print(f"Unique Vehicles: {len(get_unique_vehicles(trips))}")
    print(f"Total Distance: {total_distance(trips):.2f} km")
    print(f"Total Fuel Consumed: {total_fuel_consumed(trips):.2f} L")
    print(f"Total Fuel Cost: ₹{total_fuel_cost(trips):.2f}")


    ranking = rank_vehicles_by_efficiency(trips)


    if ranking:
```

```
    print("\n" + "-" * 80 + "\nVEHICLE EFFICIENCY RANKING\n" + "-" * 80)
    for i, (vehicle, data) in enumerate(ranking, 1):
        print(f"{i}. {vehicle} - {data['mileage']:.2f} km/L")
    print(f"\nMost Efficient: {ranking[0][0]} ({ranking[0][1]['mileage']:.2f} km/L)")
    print(f"Least Efficient: {ranking[-1][0]} ({ranking[-1][1]['mileage']:.2f} km/L)")

abnormal = find_abnormal_trips(trips)
print("\n" + "-" * 80 + "\nABNORMAL TRIP ANALYSIS\n" + "-" * 80)

if abnormal:
    print(f"Number of abnormal trips: {len(abnormal)}")
    for trip in abnormal:
        print(f"Trip {trip['trip_id']} - {trip['vehicle_no']} - {trip['mileage']:.2f} km/L")
else:
    print("No abnormal trips detected.")

duplicates = find_duplicate_trip_ids(trips)
print("\n" + "-" * 80 + "\nDUPLICATE TRIP ID ANALYSIS\n" + "-" * 80)

if duplicates:
    print("Duplicate Trip IDs: " + ", ".join(sorted(duplicates)))
else:
    print("No duplicate trip IDs found.")

if ranking:
    vehicle, data = ranking[0]
    print("\n" + "-" * 80 + "\nTUPLE-BASED VEHICLE SUMMARY\n" + "-" * 80)
    print(create_vehicle_summary_tuple(vehicle, data["distance"], data["fuel"],
data["mileage"], data["cost"]))

print("\n" + "=" * 80 + "\n" + "END OF REPORT".center(80) + "\n" + "=" * 80)
```

APPENDIX B – SAMPLE CSV DATA

B.1 vehicles.csv

```
vehicle_no,model,fuel_type,registration_date
TN01AB1234,Swift,Petrol,2026-01-10
TN02CD5678,Bolero,Diesel,2026-01-15
TN03EF9012,City,Petrol,2026-02-05
```

B.2 trips.csv

```
trip_id,vehicle_no,date,start_odometer,end_odometer,distance,fuel_filled,fuel_price
1,TN01AB1234,2026-08-01,10000,10250,250,20,102.50
2,TN02CD5678,2026-08-02,15000,15180,180,25,94.00
3,TN03EF9012,2026-08-03,20000,20600,600,35,103.00
```

The data shown above is only illustrative. The actual submission should use the real dataset that was tested against, with any synthetic or placeholder data clearly labelled as such.